Database

DISC Profiles

2025

10 December

Written by:

Harun Hursid

Betul Kazanasmazoglu Turkoglu

1. Introduction **3**
1.1. Problem description (+ architecture schema of the whole system) 3
1.2. Explanation of choices for databases and programming languages, and other tools. 4
2. Relational database **5**
2.1. Intro to relational databases 5
2.2. Database design 5
2.2.1. Entity/Relationship Model (Conceptual -> Logical -> Physical model) 7
2.2.3. Normalization process 10
2.3. Physical data model 11
2.3.1. Data types 11
2.3.2. Primary and foreign keys 12
2.3.3. Indexes 12
2.3.4. Constraints and referential integrity 13
2.4. Stored objects – stored procedures / functions, views, triggers, events 14
2.4.1 Triggers 14
2.4.2 Events 15
2.4.3 Stored Procedures 17
2.4.4 Stored Functions 19
2.4.5 Views 20
2.5. Security. 21
2.5.1. Explanation of users and privileges 21
2.5.2 Authentication and authorization 22
2.5.3. SQL Injection – what is it and how is it dealt with in the project? 24
2.6. Description of the CRUD application for RDBMS 25
2.6.1 Env files 26
2.6.2 Graphical schema 27
2.6.3 Transactions 28
2.6.4 Database backups 30
2.6.5 Migration 30
3. Document database **33**
3.1. Intro to document databases 33
3.2. Database design + features like indexes, PKs, constraints, etc. 33
3.3. Description of the CRUD application for the document database 35
4. Graph database **37**
4.1. Intro to Graph databases 37
4.2. Database design + features like indexes, PKs, constraints, etc. 38
4.2.1 Primary keys 39
4.2.2 Indexes 40
4.2.3 Constraints 40
4.3. Description of the CRUD application for the graph database 41
5. Cloud **42**
6. Conclusion **43**
7. References **45**

1. Introduction

1.1. Problem description (+ architecture schema of the whole system)

Our project focuses on improving collaboration and efficiency within organizations through the use of DISC profiles. The DISC model categorizes behavior into four main profiles Dominance (D), Influence (I), Steadiness (S), and Conscientiousness each representing a unique way of approaching tasks, communication, and teamwork. While individuals often display a mix of these traits, no two people are exactly alike, which can lead to both synergies and conflicts in the workplace[1].

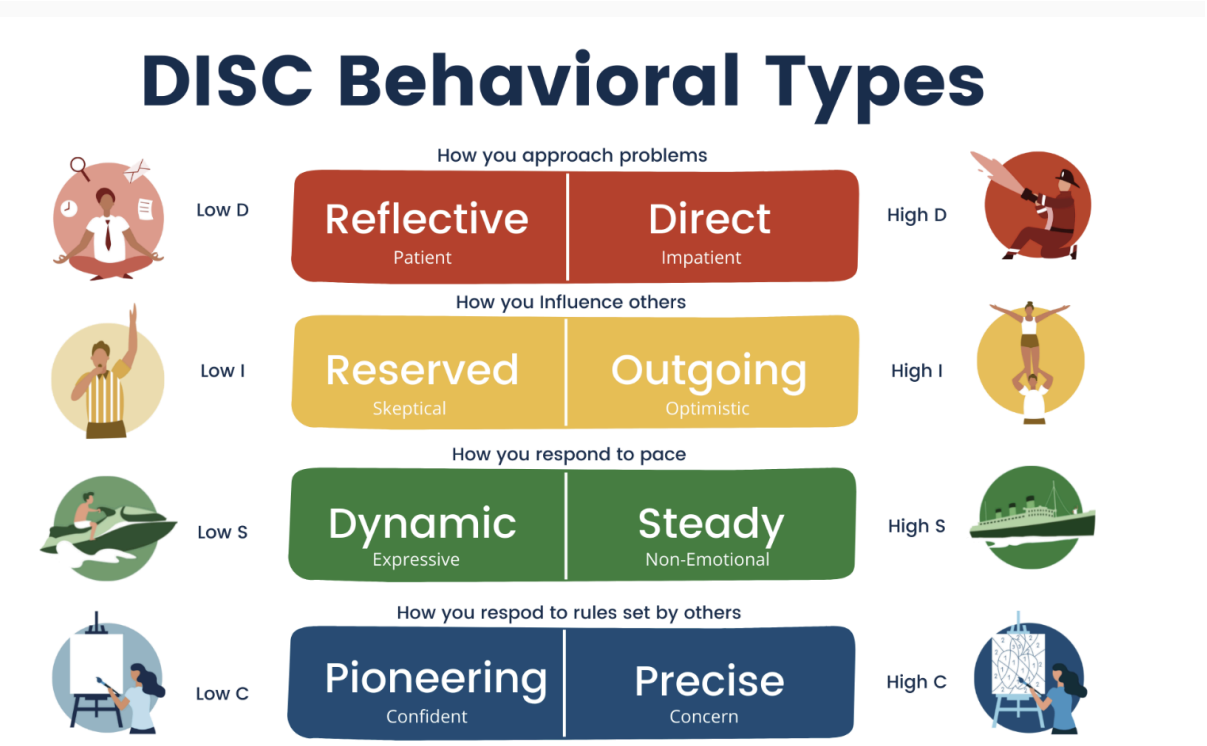

Figure 1: Short example of DISC profiles

This variability influences how employees interact. In some cases, complementary profiles lead to effective collaboration and stronger teams. In other cases, conflicting profiles may result in misunderstandings or reduced productivity. For example, a highly dominant (red) profile might clash with another dominant individual, while they may instead benefit from working with a conscientious (blue) colleague whose structured and detail-oriented approach offers balance.

[1] https://www.discprofile.com/disc-styles

Our system aims to analyze employee DISC profiles and map them to both colleagues and specific tasks. By doing so, it can provide organizations with valuable insights: how to build well-balanced teams, prevent dysfunctional dynamics, and assign tasks in a way that leverages employees' strengths. Additionally, the system can suggest collaboration strategies when employees encounter difficulties. For instance, if an individual is struggling with a task, the system can recommend a colleague with a complementary profile to provide support. Ultimately, the goal is to enhance teamwork, reduce conflict, and improve overall organizational performance and productivity.

1.2. Explanation of choices for databases and programming languages, and other tools.

We initially decided to use draw.io to create our conceptual and logical ERD schema, and chose Workbench together with MySQL. The main reason for this choice was that it seemed easiest to follow along with the lectures, where the same tools were being used. We successfully created both the logical models and a somewhat physical model in draw.io. However, after about a week, we decided to switch to Lucidchart and use Microsoft SQL Server instead of MySQL. The reasoning behind this change was first, Lucidchart makes it easier to export entities directly into SQL, which streamlined our workflow, and second, we had also chosen to implement our application in C#, making Microsoft SQL a more natural fit for our development stack.

To manage and interact with our database, we used DataGrip, which provided a more flexible and user-friendly environment and MsSQL since we thought using separate databases could make us learn about the difference, and therefore making us more knowledgeable about what to use in the future. In addition, we made use of Docker containers to ensure consistency across different environments. Running SQL server in a Docker container allowed us to create an isolated setup, avoiding dependency issues and making the system easier to share among the group. For development purposes, we relied on a local containerized environment, which ensured that each team member had the same configuration and could test features without affecting others.

2. Relational database

2.1. Intro to relational databases

A relational database is a structured way of storing and organizing data based on the principles. The central idea is that the data is represented in tables consisting of rows and columns. The relationship between the entities can be defined through keys[2].

The design process of a relational database is typically divided into four main stages: Requirements analysis, conceptual model, logical model, and physical model. In the initial phase, we conducted internal discussions and brainstorming sessions to identify which entities made sense to include, which should stand alone, and which could logically be connected to others. Since our topic DISC profiles within organizations can potentially cover a vast range of information, the scope of the database could easily become very large. This is due to the extensive data that can exist about both employees' private and professional information and the company itself, as well as details related to DISC profiling.

To keep the system manageable and aligned with the scope of the assignment, we decided to work with 13 entities as a starting point, meeting the minimum requirement of the project. Additional entities could be added later if necessary, depending on how the system developed.

2.2. Database design

The purpose of our database is to store and manage information related to employees, person profiles, and task assignments. It serves as the foundation for analyzing collaboration patterns within an organization and generating recommendations based on DISC compatibility. The database is designed to ensure both efficiency and data integrity, while remaining flexible enough to handle future expansion of entities and relationships.

The design process was based on the four classical stages of relational database development, requirements analysis, conceptual design, logical design, and physical implementation. During the initial phase, we conducted brainstorming sessions to identify relevant entities, their relationships, and how they could best represent the business logic of the DISC

[2] https://www.ibm.com/think/topics/relational-databases

framework. Since the DISC model can include extensive data about employees, teams, and company structures, we also had to consider GDPR regulations when determining which information could be stored and who would have access to it, although we haven't made login with different roles to our system yet.

Although our intention was to begin with the conceptual model, we mistakenly started by developing the physical model. This approach caused some challenges later, as it introduced technical details and join tables too early in the process, thus making it hard to read, when we started to make the other models. When we later created the conceptual and logical models, we realized that several of the tables we had initially added were unnecessary, particularly join tables that only contained foreign keys and no actual attributes, thus not making it business oriented.

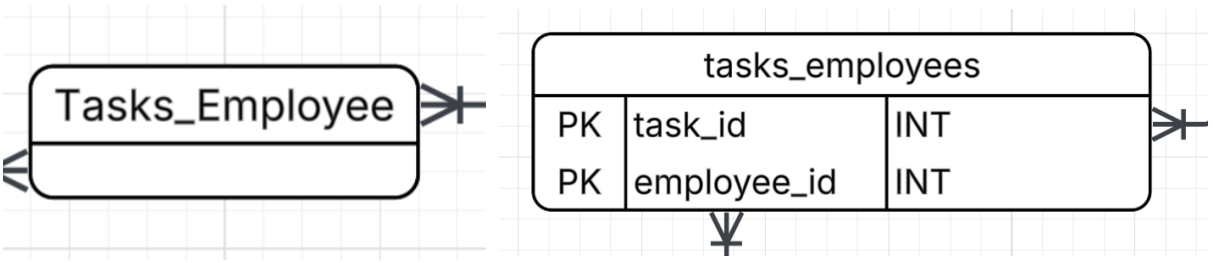

Figure 2: Conceptual and logical model example

After identifying this issue, we revisited our ERD and simplified its structure. We removed redundant join tables and focused on clearly defining the core entities and their direct relationships. This improved both readability and data consistency. Once the conceptual and logical models were finalized, we carefully reintroduced the necessary join tables in the physical model, ensuring that each had a clear purpose and followed proper naming.

Our final ERD consists of 13 main entities, including Employee, person, Company, Task, and DISCProfile, as well as a few relationship tables to handle many-to-many connections. The relationships between entities were defined with appropriate cardinalities (one-to-one, one-to-many, many-to-many) to reflect the real-world interactions of the system.

Every table includes a primary key to uniquely identify records, while foreign keys maintain referential integrity between related entities. Additional constraints, such as NOT NULL and UNIQUE are going to be implemented in the future to protect data.

The database was implemented using Microsoft SQL Server, managed through DataGrip and MSSQL, and hosted in Docker containers for local development. Docker provided an isolated and reproducible environment that allowed each team member to test features without affecting others, ensuring stability and consistency across setups.

Through this process, we learned the importance of following the correct order of database modeling, first by starting with a conceptual model before moving into technical implementation. This not only made our design more structured and easier to maintain but also strengthened our understanding of how business requirements can be effectively translated into a relational database system, therefore creating a system that meets the requirements for the business and for the developers. It also affected our way of fully understanding the system on the user side, and making sure to simplify it.

2.2.1. Entity/Relationship Model (Conceptual -> Logical -> Physical model)

If we take a look at our relational database, in figure 3 below shows our conceptual diagram, which consists of 13 entities. These entities were identified based on the business requirements to ensure that only the most relevant elements were included and that their relationships accurately reflected the real-world interactions within an organization.

In the conceptual model, we focused on representing how the different entities are connected and interact with one another. Two different types of relationship lines were used to clearly distinguish the nature and direction of each connection, making the diagram more structured and visually understandable, here we used the chen ERD notation, to specify if it is a mandatory or an optional relationship between the two entities. The purple text in the diagram serves as descriptive relationship labels, which make it easier to read the connections between entities such as Employee, Project, Tasks, and DISC Profiles. This helps users quickly identify how the business logic translates into relationships in the database.

We then used the Crow’s foot notation on the cardinality (relationships) between the entities. We used many to many, one to many and one to one relationships to identify how many employees and be part of a project and see if a project can exist without employees, which in this case it can not.

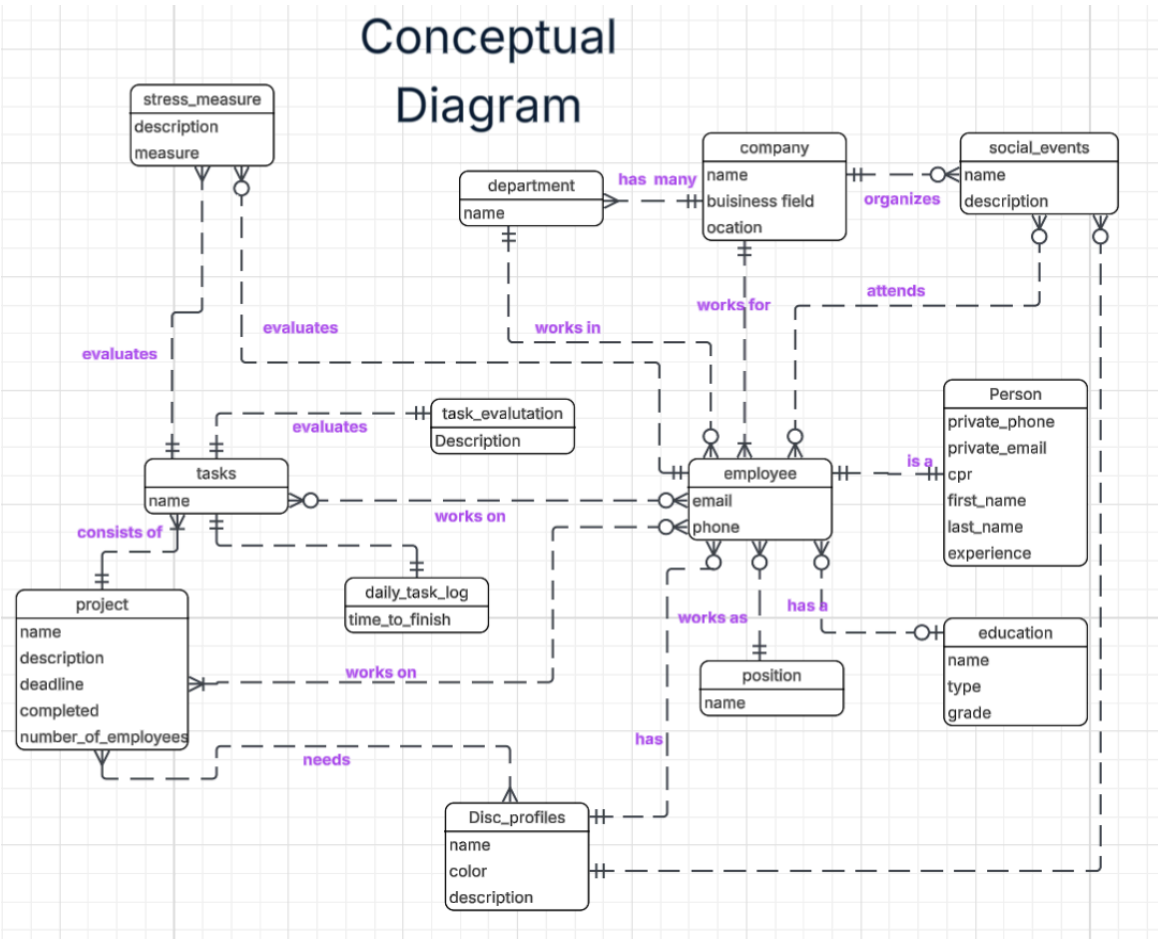

Figure 3: ERD Conceptual model

In our logical model as seen on figure 4, there is a streamflow between the conceptual model but with a slight difference. The relationships are for the most part optional but with some few mandatories. For example the relationship from employee to social event, this indicates that there can't be an event without employees, or else the company used money from the budget on an event that nobody came to, same goes to the employee to task entity. There can’t be any task without any employees, since the task can’t be completed, but there can be tasks within the project existing and waiting for one or many employees to join and finish. Some tables in the model are empty which in this case indicates that these entities have data in them that aren't new or descriptive attributes, these entities only have primary and foreign keys in them, therefore not important to show which can be seen on the physical model.

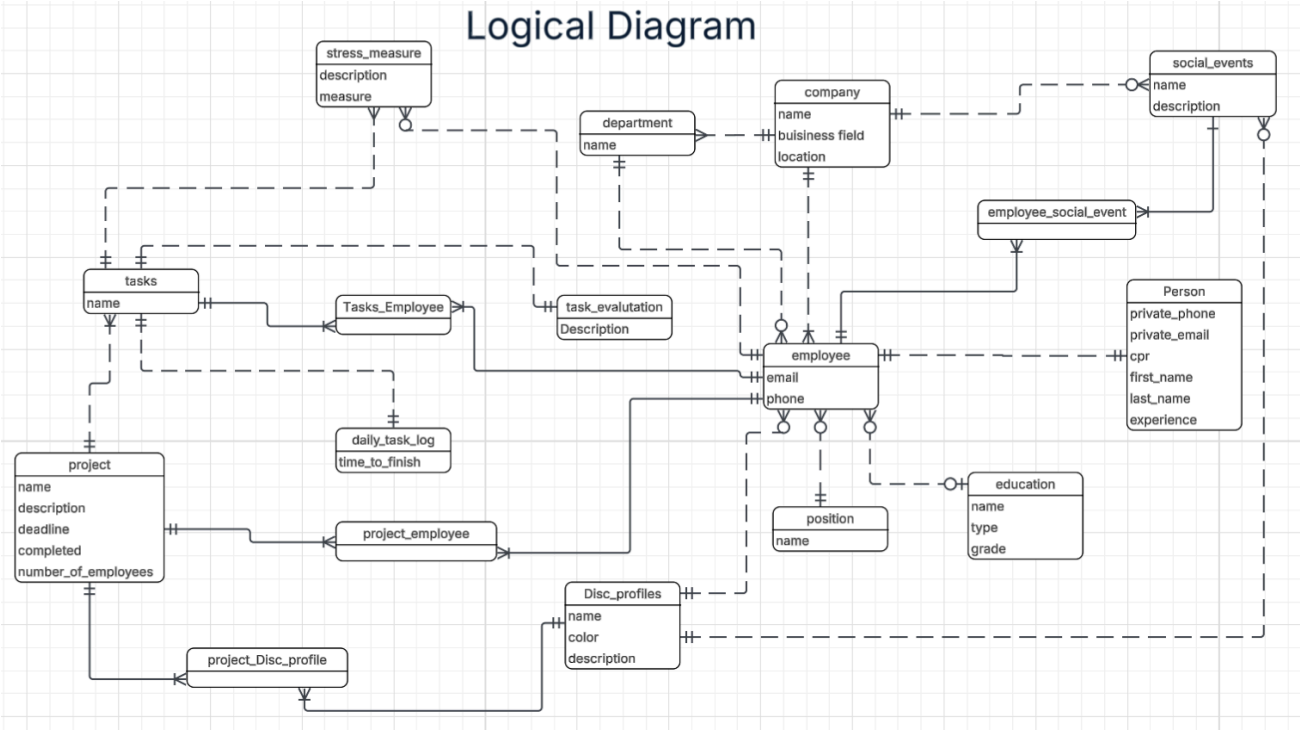

Figure 4: ERD Logical model

In our physical model, as seen in Figure 5 below, we continued using Lucidchart to manually define the relationships, entities, attributes, and data types. This model also includes entities with key attributes, where only the primary and foreign keys are necessary to store and connect data. This was also the reason why we ended up creating more entities than the initial 13, but only creating for now join tables.

The join tables were introduced to handle many-to-many relationships between entities. They link two entities together by storing their keys in a separate table instead of duplicating the same data across multiple tables. This approach maintains data consistency, and makes the database more flexible for future development, allowing us to easily add additional attributes to these relationships if needed later on. The relationships continue to stay the same, making the workflow/flow of the project continue and similar.

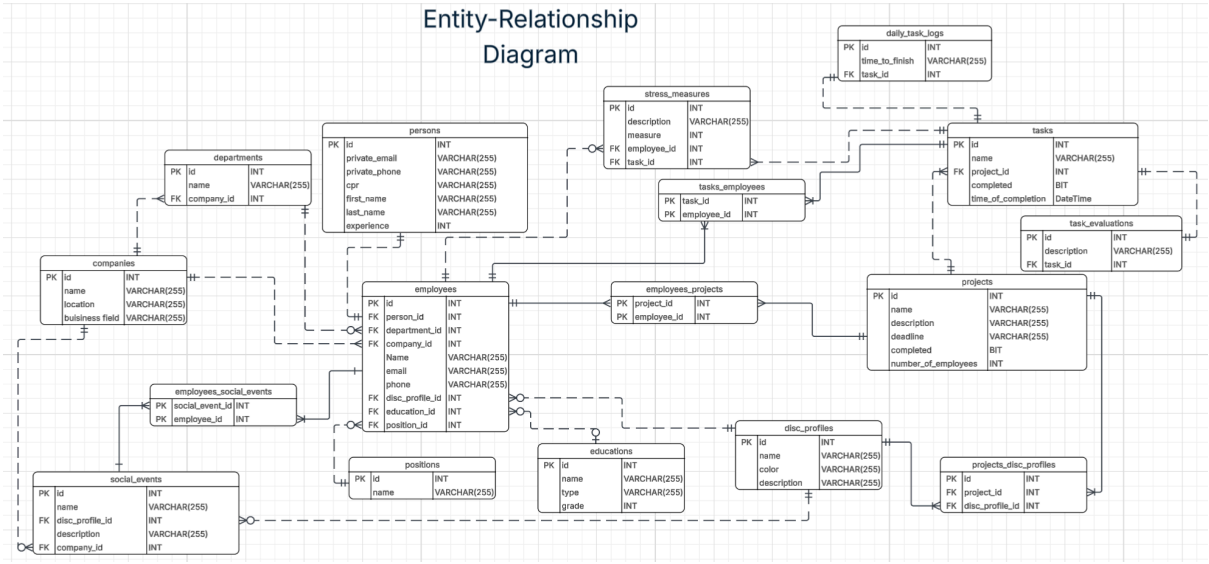

Figure 5: ERD physical model in lucidchart

2.2.3. Normalization process

Normalization is a set of rules used when designing a relational database. The purpose is to make the database easier to maintain and to avoid redundant data. These rules are called normal forms, and each normal form can only be met if the previous one is satisfied. In our design, the normalization rules directly influenced how we structured our tables and relationships.

To satisfy 1NF, all attributes must contain single atomic values and no tables may contain repeating groups. In our database, this requirement led us to create a separate table for each entity and to avoid storing lists inside a single row. For example a project can have many employees, and an employee can work on many projects. Instead of storing a list of employees in the projects table, we created the join table employees_projects. As seen on figure 6 all entity attributes such as name, deadline and description contain only a single value, meaning the entire design complies with 1NF.

Figure 6: Join table from employees_ projects to projects

2NF applies to tables with composite primary keys and requires that any non-key attributes depend on the entire composite key. The rule states that every non-key attribute must depend on the entire composite key, not just part of it.
In our database, composite keys appear only in join tables such as, employees_project, tasks_employees and employees_social_events. These tables contain no additional attributes, meaning that all their columns depend entirely on the full composite key. As a result, these tables automatically satisfy 2NF. For example, if we were to assign attributes such as hours_spent for a specific employee on a specific task, these would be placed in the relevant join table because they depend on both parts of the composite key. Because hours_spent is not just about the tasks and not just about the employee, it describes the combination of a specific employee working on a specific task.

In 3NF requires that all non-key attributes depend only on the table's primary key and not on another non-key attribute meaning there must be no transitive dependencies. In our design, the most important 3NF decision concerns the employees' tables. We choose not to store information such as department_name, education_name, positions_name etc. directly inside the employees table.

There is one table which does not satisfy the 3NF. The projects table contains the attribute number_of_employees. This value is not an inherent property of the project itself but is instead derived from counting rows in the employees_projects table. Because this attribute depends on data in another table, it introduces redundancy and the risk of inconsistencies, which is also a violation of 3NF.

2.3. Physical data model

2.3.1. Data types

For our data types in our database we have primarily used varchar(255) that is like a string in database language. We have used that in most of our text based values. If you notice in the ‘Person’ table there is a record of the Danish identification number (CPR) that could easily be an int but we choose to use varchar instead because if we made an int and the CPR started with an 0 it would delete that 0 and that would give us a problem later on, that's why we choose to use it as a varchar. The second datatype is primarily int for all of our numerical values, and our keys.

We have a third data type we use, ‘bit’ if you see in our ‘Task’ and ‘projects’ you can see the record completed is of data type ‘BIT’ which is basically a boolean but with 0-1, so 0 means false and 1 means true, we use it to see if a task or project is completed. And the last datatype we have used is datetime, in the table ‘Tasks’ we have a record ‘Time_of_Completion’ we chose for it to be a datetime, so it is exactly the date and time of completion which is important for us to track when a task or project is done in order to track the stress level for an employee or multiple employees. We can also track the productivity of an employee or group of employees based on the time required for the task, so for example if a task is normally 1-2 hours completion time for an average employee but the assigned employee uses maybe 3-4 hours it could be a hint of stress.

It could also improve productivity by assigning employees that excel in the area required.

2.3.2. Primary and foreign keys

We have a lot of primary and foreign keys in our database, we use primary keys in all our tables to identify what is the id of the record inside the table. It also helps us sort and find data quickly by searching on id’s and setting parameters based on id, if we use functions or stored procedures.

We use foreign keys to link some tables with other tables by creating a relationship and linking the tables on foreign keys. It creates a parent - child relationship so then we know the relationships the tables have to each other.

2.3.3. Indexes

Indexes improve read performance by speeding up queries that search, filter, or join data. SQL server automatically creates a clustered index for primary keys unless otherwise specified. A clustered index determines the physical order of rows in the table. Nonclustered indexes store a separate copy of the indexed columns along with pointers to the corresponding rows. This allows much faster lookups, but it also creates redundant data. Because the database must update both the base table and the index, a large number of nonclustered indexes can slow down write operations such as INSERT and UPDATE.

In our design, we created nonclustered indexes on all foreign key columns. These columns are frequently used in join operations (for example, joining employees to companies or tasks

to projects), and indexing them allows SQL Server to perform an index seek rather than scanning entire tables. When a query needs additional columns not covered by the index, SQL Server performs a key lookup, which is still much faster than a full table scan.

```sql
-- Create indexes for foreign keys to improve query performance
CREATE INDEX idx_departments_company ON departments(company_id);
CREATE INDEX idx_social_events_disc_profile ON social_events(disc_profile_id);
CREATE INDEX idx_social_events_company ON social_events(company_id);
CREATE INDEX idx_employees_person ON employees(person_id);
CREATE INDEX idx_employees_department ON employees(department_id);
CREATE INDEX idx_employees_company ON employees(company_id);
CREATE INDEX idx_employees_education ON employees(education_id);
CREATE INDEX idx_employees_disc_profile ON employees(disc_profile_id);
CREATE INDEX idx_employees_position ON employees(position_id);
CREATE INDEX idx_tasks_project ON tasks(project_id);
CREATE INDEX idx_project_disc_profile_project ON projects_disc_profiles(project_id);
CREATE INDEX idx_project_disc_profile_disc_profile ON projects_disc_profiles(disc_profile_id);
GO
```

Figure 7: Indexes

In addition to the indexes we created, it is important to note that our design does not include indexes on certain frequently filtered columns, most notably completed in both the tasks and projects tables. Queries and triggers that check whether a task or project has been completed could benefit from such indexes. However, we intentionally avoided indexing these columns because they are updated frequently, and indexing them would introduce additional write overhead. At the current scale of our dataset, this design decision has minimal performance impact.

Overall, the indexes we created allow SQL Server to use index seeks for most join operations, minimizing the need for full table scans and improving responsiveness. The few indexes we did not include represent optional optimizations that can be introduced later if the database grows significantly or if specific query bottlenecks appear.

2.3.4. Constraints and referential integrity

Constraints is how we store our data and validate our inputs so that means when we use a primary key it is unique identification in that table. We make sure the values inside the records are not null and assuring foreign keys to create relation between tables.

In our database we made sure that the fields that should not be null are not null by asserting not null when creating the record.

We also made sure that when creating the table and the primary ID we made sure its identity 1-1. That means every time a new record is created it handles the ID for us so we don't have to insert it, it just adds on top of the previous one.

Referential integrity makes sure that every time you referrer by a foreign key it makes sure that whatever your referrer to exists. So for example if 'employees' have a foreign key to company ID it has to make sure the company exists otherwise it creates a parentless record which could lead to the data not referring to the right place in a logical way.

2.4. Stored objects – stored procedures / functions, views, triggers, events

2.4.1 Triggers

Triggers are blocks of SQL code that automatically execute in response to changes in a table, such as an INSERT, UPDATE, or DELETE. They are typically used to enforce business rules or maintain data integrity without relying on application logic.

In our database, the tasks table includes a column called completed, which indicates whether a task is finished. To automate this process, we created an AFTER INSERT trigger on the daily_task_logs table. When a user logs the time spent on a task, the trigger automatically updates the corresponding task and sets completed = 1. This ensures that the system reflects the status of the task without manual updates.

```
CREATE  TRIGGER task_is_complete ON daily_task_logs
    AFTER INSERT
    AS
    BEGIN
        -- we don't want to see (1 row(s) effected):
        SET NOCOUNT ON;
        UPDATE t
        SET completed = 1
        FROM tasks t
        INNER JOIN inserted i ON t.id = i.task_id
        WHERE t.completed = 0;
    END;
go
```

Figure 7: Trigger example

We also created an INSTEAD OF INSERT trigger on the same table to maintain data integrity. This trigger checks whether the task associated with the new log entry is already completed. If the task has already been marked as completed, the trigger prevents the insertion and raises a custom error. This ensures that no new time entries can be added for tasks that are finished, avoiding inconsistent or misleading data.

Together, these triggers automate task completion and enforce data correctness directly at the database level.

```sql
-- check tasks table to see if the task is already completed
-- if it is you cant make daily_task_logs entry
CREATE TRIGGER only_log_incompleted_tasks ON daily_task_logs
    INSTEAD OF INSERT
    AS
    BEGIN
        SET NOCOUNT ON;
        IF EXISTS (
        SELECT 1
        FROM inserted i
        INNER JOIN tasks t ON i.task_id = t.id
        WHERE t.completed = 1
    )
    BEGIN
        -- Throw an error and stop execution
        THROW 50001, 'Cannot log time for completed tasks. The task is already marked as completed.', 1;
        RETURN;
    END
        INSERT INTO daily_task_logs (time_to_finish, task_id)
        SELECT i.time_to_finish, i.task_id
        FROM inserted i
        INNER JOIN tasks t ON i.task_id = t.id
        WHERE t.completed = 0;
    end
```

Figure 8: Triggers

2.4.2 Events

Events are similar to triggers in that they also execute automatically, but instead of reacting to data changes, they run according to a predefined schedule. While MySQL provides event functionality directly, Microsoft SQL Server handles scheduled tasks through the SQL Server Agent, which can execute jobs at specific intervals.

A SQL Server Agent job consists of one or more job steps, each capable of running T-SQL statements, PowerShell scripts, or other actions. Jobs and their schedules are stored in the msdb system database, which is part of every SQL Server installation.

In our project, we created a scheduled job that runs once per day at 13:25. This job checks the projects table for entries where: completed = 1

the project's deadline is more than one year old

If such projects exist, the job deletes them. This prevents outdated completed projects from staying in the main dataset indefinitely and helps maintain a clean and relevant database. Jobs can be configured entirely using T-SQL or through SQL Server Management Studio's graphical interface. Multiple jobs can share the same schedule, and a single job may have several schedules attached.

```sql
USE msdb;
GO
-- 1. Create the job
EXEC sp_add_job
    @job_name = N'Delete Completed Projects Older Than 1 Year';
GO
-- 2. Add a job step to perform the deletion
EXEC sp_add_jobstep
    @job_name = N'Delete Completed Projects Older Than 1 Year',
    @step_name = N'DeleteOldProjects',
    @subsystem = N'TSQL',
    @command = N'
        DELETE FROM [my_disc_profiles_playground].dbo.projects
        WHERE completed = 1
          AND TRY_CAST(deadline AS DATE) < DATEADD(YEAR, -1, GETDATE());
    ',
    @database_name = N'my_disc_profiles_playground';
GO

-- 3. Create a schedule to run daily at 13:25
EXEC sp_add_schedule
    @schedule_name = N'sched_name',
    @freq_type = 4,              -- Daily
    @freq_interval = 1,          -- Every day
    @active_start_time = 112500; -- At 13:25(its running a different timezone)
GO

-- 4. Attach the schedule to the job
EXEC sp_attach_schedule
    @job_name = N'Delete Completed Projects Older Than 1 Year2',
    @schedule_name = N'sched_name';
GO

-- 5. Assign job to SQL Server
EXEC sp_add_jobserver
    @job_name = N'Delete Completed Projects Older Than 1 Year2';
GO

PRINT 'Scheduled job created successfully!';
GO
```

Figure 9: Events

In a production system, it would be more efficient to schedule cleanup jobs during periods of low activity, such as overnight. Instead of permanently deleting old projects, many systems would archive them into a dedicated archive table for historical or analytical purposes. Furthermore, this type of maintenance job does not need to run daily; running it once per month would often be sufficient.

2.4.3 Stored Procedures

A stored procedure is a saved function inside your database to execute a number of methods that you write to get more complex data from your database. I will give an example of what we did in our database. We wrote a stored procedure that gives an overview over a company.

The script starts off by selecting a company. We select it by ID as shown below:

```
-- 1) Virksomhed
SELECT *
FROM companies
WHERE id = @CompanyId;
```

Figure 11: Stored procedures

Then we select the department of the company so what department belongs to the selected company:

```
-- 2) Afdelinger
SELECT *
FROM departments
WHERE company_id = @CompanyId;
```

Figure 12: Stored procedures

Then we select employees that work at the departments of the chosen company:

```
-- 3) Medarbejdere (inkl. afdelingsnavn)
SELECT e.*,
       d.name AS department_name
FROM employees e
LEFT JOIN departments d ON d.id = e.department_id
WHERE e.company_id = @CompanyId;
```

Figure 13: Stored procedures

We use a left join to get the name of the department that the employee is working from. We then find projects that the employees from the chosen company work on and again we use an inner left join to get the data from the employee. We also get a calculated table where we get

team size of the project.

```sql
-- 4) Projekter med mindst én medarbejder fra virksomheden
--    + afledt felt: team_size
SELECT
    p.*,
    (
        SELECT COUNT(*)
        FROM employees_projects ep
        WHERE ep.project_id = p.id
    ) AS team_size
FROM projects p
WHERE EXISTS (
    SELECT 1
    FROM employees_projects ep
    INNER JOIN employees e
        ON e.id = ep.employee_id
    WHERE ep.project_id = p.id
      AND e.company_id = @CompanyId
);
```

Figure 14: Stored procedures

```sql
-- 5) Sociale events for virksomheden
SELECT se.*
FROM social_events se
WHERE se.company_id = @CompanyId;
```

Figure 15: Stored procedures

We get all the social events connected to the selected company.

	id	name	location	business_field
1	1	Tech Solutions Inc	Copenhagen	Information Technology

	id	name	company_id
1	1	Software Development	1
2	2	Human Resources	1

	id	name	email	phone	company_id	person_id	department_id	education_id	department_name
1	1	Anders Nielsen	anders.nielsen@techsolutions.com	+45 20 12 34 56	1	1	1	1	Software Development
2	2	Maria Jensen	maria.jensen@techsolutions.com	+45 30 23 45 67	1	2	1	2	Software Development
3	3	Lars Hansen	lars.hansen@techsolutions.com	+45 40 34 56 78	1	3	2	3	Human Resources
4	13	Martin Madsen	martin.madsen@techsolutions.com	+45 61 34 56 78	1	13	1	1	Software Development

	id	name	description	deadline	completed	number_of_employees	team_size
1	1	E-commerce Platform	Building new online shopping platform	2025-12-31	No	5	3
2	6	Website Redesign	Company website overhaul	2025-08-31	Yes	2	2
3	7	Data Analytics Platform	Business intelligence dashboard	2025-10-31	No	4	3

	id	name	disc_profile_id	description	company_id
1	1	Summer BBQ Party	2	Annual company barbecue event	1
2	2	Team Building Workshop	3	Collaborative problem-solving activities	1
3	8	Company Anniversary ...	2	10-year anniversary celebration	1

Figure 16: Stored procedures

This is the final result for the stored procedure. It's all the necessary information about a company and its departments,employees and their assigned department, projects and the social events.

2.4.4 Stored Functions

Stored functions are code in sql that lets you reuse logic and returns a result. So instead of writing complicated joins and long sql statements repeatedly you can create a function where all the complicated code is and simply call the function anytime you need it. in sql there are 2 types of functions, Scalar functions that only return a value for example a number or text. and the other one is a Table-valued function that returns a whole table with multiple records, kinda like a mini view with parameters.

In our project we have made a Table-valued function that finds all the projects a given employee is working on. Our function joins on the projects table to get all the details of a project and then it is filtering on employees to match the given employee.

```sql
CREATE OR ALTER FUNCTION dbo.fn_EmployeeProjects (@employeeId INT)
RETURNS TABLE
AS
RETURN
(
    SELECT p.id AS project_id,
           p.name AS project_name,
           p.deadline,
           p.completed
    FROM employees_projects ep
    JOIN projects p ON p.id = ep.project_id
    WHERE ep.employee_id = @employeeId
);
GO
```

Figure 17: Stored functions

Then the final result of the function is a table with all the projects that the employee is working on.

project_id	project_name	deadline	completed
1	E-commerce Platform	2025-12-31	No
6	Website Redesign	2025-08-31	Yes

Figure 18: Stored functions

2.4.5 Views

A view is a saved SELECT query that you can treat like a table because you simply query it like a normal table after it's created. The view uses data from one or multiple tables and every time you execute the view it simply runs the sql query. The data shown in the view is always based on the underlying tables.

We have created a view in our database that shows the results of social events and what disc profiles match the events.

This is how we created the view, we select from the social event table, company and discprofile and then use a left join to get the data.

```sql
CREATE OR ALTER VIEW vw_SocialEventsOverview
AS
SELECT
    se.id AS social_event_id,
    se.name AS event_name,
    se.description AS event_description,
    c.name AS company_name,
    dp.name AS disc_profile_name,
    dp.color AS disc_color,
    dp.description AS disc_description
FROM social_events se
LEFT JOIN companies c ON se.company_id = c.id
LEFT JOIN disc_profiles dp ON se.disc_profile_id = dp.id;
GO
```

Figure 19: Views

This is how to execute the view:

```sql
SELECT * FROM vw_SocialEventsOverview;
```

Figure 20: Views

2.5. Security.

2.5.1. Explanation of users and privileges

Since we did not implement users and privileges in the initial version of the database, we first had to revisit the design and ensure that the models supported a proper security structure. Our goal was to introduce clear permission boundaries so that different types of users only have access to the data and operations they actually need.

In our setup we distinguish between an admin user and a normal user. On the application level, the admin has full access to almost all entities through the backend: admins can create, update and delete users and employees, and in general manage the data that defines the projects and the organisation. This role is intended for system administrators and developers who are responsible for maintaining the system and its data. The normal user has more limited permissions. A normal user is allowed to read general information about themselves and manage their own personal information, such as phone number or email. However, they cannot modify company-wide information, access data about other employees or change system-level configuration. This reduces the risk that regular users accidentally or intentionally perform critical changes to the system.

Instead of relying on the SQL server users and privileges, we decided to centralise all access through our ASP.NET Web API. We did this because our application is a web-based system where all database access goes through that. We choose to implement users, roles and privileges at the application layer instead of using individual SQL server logins. This approach matches how many real world systems are built. The Database is not exposed directly to end users, and role based security is handled in the backend, where we have full control over business rules and can combine information from multiple tables.

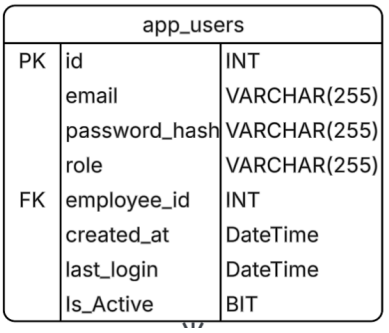

Figure 21: New table in ERD diagram

As part of this redesign we introduced an app_user table that represents the accounts that can log in to the system see figure 21. This required updating our ER diagram to include AppUser and its foreign key to Employee. The link between AppUser and Employee is important for our privilege model, because it clearly separates “who is logged in” (the AppUser account) from “which employee they represent” (the Employee and related Person entities). The link between AppUser and Employee is important for our privilege model. It allows us to see the difference between who is logged in and which employee they represent, which is the connection between the employee table and Person table.

We initially tested the security endpoints using Swagger, but later added a small HTML frontend to make the privileges more visible. The root page links to login.html, admin.html, employee.html and me.html. A normal user can log in and view their own profile information on the “me” page and at this stage the view focuses on basic identity details such as email, employee id, role, creation time and last login. Admin users are redirected to the admin page, where they can see an overview of all users and create new accounts, and then redirect to the employee.html to update some information and connect them to a project or task..

2.5.2 Authentication and authorization

On top of the user and role model, we implemented a simple but robust authentication and authorization mechanism based on JSON Web Tokens (JWT) Authentication is handled by the AuthController through the api/auth/login endpoint. When a user submits their email and password from the login.html page, the backend looks up the corresponding AppUser by email and checks that the account is active. The provided password is then verified against the store's password hash using our PasswordHash, which uses BCrypt for hashing. This ensures that the passwords are never stored in plain text and that each password has its own salt, making it much harder to recover in case of dataleak.

```
//Hash password on registration
2 references
string HashPassword(string password);
//Verify password on login
2 references
bool VerifyPassword(string password, string hash);
```

Figure 22: Code Hash in project

If the information is correct, the backend then updates the LastLogin timestamp for the user and calls JwtTokenService to generate a signed JWT. The token contains a small set of

claims, including the user id, email and role. The frontend stores this token in LocalStorage and sends it with all the API calls via the Authorization: Bearer <token> header.

```
2 references
public bool VerifyPassword(string password, string hash)
{
    if (string.IsNullOrWhiteSpace(password) || string.IsNullOrWhiteSpace(hash))
        return false;

    try
    {
        return BCrypt.Net.BCrypt.Verify(password, hash);
    }
    catch
    {
        return false;
    }
}
```

Figure 23: JWT

```
// Add Authentication
var jwtConfig = builder.Configuration.GetSection("Jwt");
var jwtKey = jwtConfig["Key"];

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey)),
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidIssuer = jwtConfig["Issuer"],
            ValidAudience = jwtConfig["Audience"],
            ValidateLifetime = true,
            ClockSkew = TimeSpan.Zero
        };
    });
```

Figure 24: JWT

In program.cs we then configure ASP.net Core's JWT Bearer authentication to validate the token signature using a secret key from appsettings.json, as well as check issue, audience and expiration time. The system is stateless, since the token itself is the only proof that the user is logged in.

Authorization uses ASP.NET Core's built in role system. Actions that require login are marked with "authorize" and admin-only endpoint use [Authorize(Roles = "admin")]. for example, AppUserController is restricted to the admin, while AuthController only needs a valid token and returns data for the current user from the JWT claims. On the frontend, admin is redirected to the admin view, and normal users to their personal profile, and they both need to log in, to access these views or else they will be redirected to the login.html view.

```
// Hvis ingen token → tilbage til login
if (!token) {
    window.location.href = 'login.html';
}
```

Figure 25: JWT

```
  // Admin → admin.html, ellers me.html
  if (data.role === 'admin') {
      window.location.href = 'admin.html';
  } else {
      window.location.href = 'me.html';
  }
catch (err) {
  errorDiv.textContent = "Der opstod en fejl. Prøv igen.";
  console.error(err);
```

Figure 26: JWT redirect

2.5.3. SQL Injection – what is it and how is it dealt with in the project?

SQL injection occurs when user-controlled input is inserted directly into a SQL query without proper validation or parameterization. This allows a malicious user to inject and execute harmful SQL commands, which can lead to actions such as deleting data, dropping tables, or gaining unauthorized access to the database.

For SQL injection we did not actively do anything to prevent it since we found out that Entity Framework prevents SQL injection for us if used correctly, by doing parameterized queries. So in Entity Framework we use LINQ expressions to communicate with SQL.
Additionally, SQL injection is only possible when user-controlled input is inserted directly into raw SQL strings. Because Entity Framework generates parameterized commands automatically, user input is never executed as SQL code but treated strictly as data. This means that even if a malicious user attempts to inject harmful SQL, the database will not interpret it as executable commands. [3]

[3] https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/ef/security-considerations

2.6. Description of the CRUD application for RDBMS

We built the backend as a CRUD application in C# using ASP.NET Core and Entity Framework Core. Since we followed a database-first approach, the SQL database was created before the backend code. Using Entity Framework's Scaffold-DbContext command, we generated all entity classes and the DbContext directly from the database schema.

The DbContext manages the database connection, maps tables to C# classes, translates LINQ queries into SQL, and performs INSERT, UPDATE, and DELETE operations when SaveChanges() is called.

Our backend follows a layered architecture:

- Entities: Auto-generated C# classes that represent the database tables.

- Repositories: A generic repository layer responsible for all database access.

- Services: Contains business logic, at this point only CRUD, including special cases like creating an Employee + Person in one transaction and app_users + employee + person in one as well.

- DTOs: Used for clean input/output models to avoid exposing internal database structure.

- Controllers: API endpoints that receive HTTP requests, call services, and return JSON responses.

This structure provides clean separation of concerns, makes the system maintainable, and follows modern .NET best practices.

2.6.1 Env files

We decided to make use of an env file so that our connection strings were not exposed in the code. We made an empty document where we wrote our connection string as environment variables. we added the env file to gitignore so it didn't get pushed to git. The way .env files works is to download a nuget package and then in the program.cs class use env.Load() and set the connection string variable to load from Environment. Then the connectionstring variable inside the usesqlserver function.

```
Env.Load();

var connectionString = Environment.GetEnvironmentVariable("CONNECTION_STRING");

builder.Services.AddDbContext<DiscProfilesContext>(options =>
    options.UseSqlServer(connectionString));
```

Figure 27: .Env

We also did the same for our migrator app where we had 3 connection strings one for sql, one for mongodb and one for database name for mongo db

```
// 1) Load .env
Env.Load();

var sqlConnectionString = Environment.GetEnvironmentVariable("SQL_CONNECTION_STRING");
var mongoConnectionString = Environment.GetEnvironmentVariable("MONGO_CONNECTION_STRING");
var mongoDatabaseName = Environment.GetEnvironmentVariable("MONGO_DATABASE_NAME");
```

Figure 28: .Env

It looks pretty similar to the api but with additional two variables.

We have made some changes to the env files. When we deploy our databases to the cloud we need to update our environment variables to point towards the cloud so we have added for our api a local and cloud environment variable for MSSQL server so we can access the cloud database. We also have a local environment variable for MSSQL in case you need to work on something locally.

We plan to do the same for both MongoDB and Neo4j to add their Local and cloud connections in our environment file.

2.6.2 Graphical schema

This is the graphical schema of the backend. It shows how a request moves when you send a http request to the controller and through the layers and down to the database.

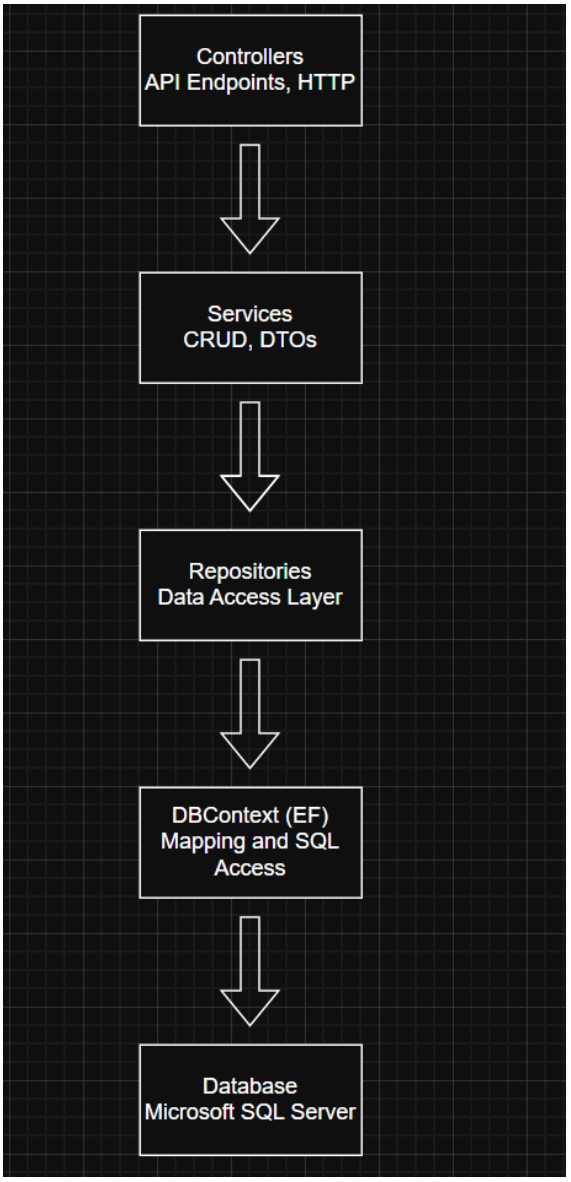

Figure 29: Graphical schema

2.6.3 Transactions

A transaction is a sequence of operations that are treated as a single unit of work. This means that if a transaction contains multiple steps, and just one of them fails, the entire transaction is rolled back. Nothing is saved to the database unless all operations succeed. In other words, either everything is executed, or nothing is executed.

In our project, we implemented two transactions in the backend. It is important to note that transactions created inside SQL Server Management Studio (SSMS) cannot be used or accessed by the backend, and backend transactions cannot be reused in SSMS either. They are completely separate, because each transaction belongs to its own connection and execution context.

One of our transactions is used when creating an Employee. In our data model, every Employee must be linked to a Person, but a Person can exist without being an Employee. Because of this dependency, creating an Employee must also create a Person in the same operation. To ensure this happens safely and consistently, we wrapped both operations inside a single transaction. The transaction looks like this

```
public async Task<EmployeeDTO> CreateEmployeeWithPersonAsync(CreateEmployee_PersonRequestDto dto)
{
    await using var transaction = await _context.Database.BeginTransactionAsync();

    try
    {
        // 1) Person via generic repo
        var person = new person
        {
            private_email = dto.PrivateEmail,
            private_phone = dto.PrivatePhone,
            cpr = dto.Cpr,
            first_name = dto.FirstName,
            last_name = dto.LastName,
            experience = dto.Experience,
            EducationID = dto.EducationId
        };

        person = await _personRepo.AddAsync(person);
```

Figure 30: Transactions

Inside the transaction, we first create the Person and then the Employee referencing that Person. If any part of the process fails, the entire transaction is rolled back, so we never end up with a Person without an Employee-reference or an Employee missing the required Person entry.

This design is particularly interesting because our entire CRUD system is built using a generic repository and service setup, which handles all standard operations such as Get, Update, and Delete. However, the Employee creation process required more complex

business logic, so we implemented a separate domain service specifically for the transaction logic.

```
// 2) Employee via generic repo
var employee = new employee
{
    email = dto.WorkEmail,
    phone = dto.WorkPhone,
    company_id = dto.CompanyId,
    person_id = person.id,
    department_id = dto.DepartmentId,
    position_id = dto.PositionId,
    disc_profile_id = dto.DiscProfileId
};

employee = await _employeeRepo.AddAsync(employee);

await transaction.CommitAsync();

return new EmployeeDTO
{
    id = employee.id,
    email = employee.email,
    phone = employee.phone,
    company_id = employee.company_id,
    person_id = employee.person_id,
    department_id = employee.department_id,
    position_id = employee.position_id,
    disc_profile_id = employee.disc_profile_id
};
```

Figure 31: Transaction

```
catch
{
    await transaction.RollbackAsync();
    throw;
}
```

Figure 32: Transaction

In the EmployeeController, we combine the generic service with our custom transaction service. All normal CRUD operations (Get, Update, Delete) use the generic service, but the Create operation uses the transaction service to ensure that both the Person and the Employee are created together in one atomic operation.

The second transaction makes sure that when an admin creates a new user that doesn't exist yet in our database, the system then creates a Person, Employee and an AppUser in a single transaction. In other words, every user who can log in is always associated with an employee and a person recorded in the schema. These two transactions make sure that when the employee is created a person is also part of it, but doesn't always have an AppUser yet

because of other relation to the company but if the company needs to have an immediate AppUser, that isn't part of the company yet, then the second transactions makes sure of this.

2.6.4 Database backups

For database backup in the beginning of our project we actually did not make any backups because we chose to run a localhost setup. That meant that both of us had a microsoft sql server on our devices and a database each for this assignment. So basically if one of our databases failed we would have had the other one as a backup, also we had all the scripts for the database to recreate it if anything failed.

But for the final exam project we noticed that it is a requirement for the database and backend to be hosted online in azure or other web services so we have obviously implemented that. The way the implementation for the database backup was made was that we exported the database to a '.Bak' file so we can recreate the database if anything goes wrong. We can recreate the database with all the tables, views and data etc.

We also hosted our databases on an online hosting service. For MSSQL we used Simply.com and to export the database to simply.com we had to upload the '.Bak' file to recreate the database online. We also published the Backend-api at simply.com for browser use so everything can be in the cloud. We had a lot of trouble with azure when we tried to create a web app, we found out that using an azure student account restricted our use of the service, so we decided to look elsewhere, and use other services.

To use Simply.com we had to buy a domain called DiscProfiles.dk, and pay for it continuously. Other services like render.com, couldn't host our database, since we did not use Postgress, therefore making the decision to only use simply.

To upload the api we had to publish via visual studio to a file and use filezilla to upload all the files to our Simply.com .

2.6.5 Migration

We created a migration app inside the same solution as the api but we just added another project this way it is in the same folders and everything plus we have it in the same github repository for easier use.

But basically we created a console app in C# with only a program.cs class that holds all the

logic. We used as mentioned before an env file for the migration as well for all the connection strings to the databases.

The way we created the migrator was by starting to load the environment variables

```
// 1) Load .env
Env.Load();

var sqlConnectionString = Environment.GetEnvironmentVariable("SQL_CONNECTION_STRING");
var mongoConnectionString = Environment.GetEnvironmentVariable("MONGO_CONNECTION_STRING
var mongoDatabaseName = Environment.GetEnvironmentVariable("MONGO_DATABASE_NAME");

* if (string.IsNullOrWhiteSpace(sqlConnectionString) ||
    string.IsNullOrWhiteSpace(mongoConnectionString) ||
    string.IsNullOrWhiteSpace(mongoDatabaseName))
{
    Console.WriteLine("❌ Missing environment variables. Check .env.");
    return;
}*/

if (string.IsNullOrWhiteSpace(sqlConnectionString))
{
    Console.WriteLine("check sql connection string");
    return;
}
if (string.IsNullOrWhiteSpace(mongoConnectionString))
{
    Console.WriteLine("check mongo connection string");
    return;
```

Figure 33: Migration

by doing this we also check for any missing variables if one or more is missing the migrator stops.

```
private static async Task MigrateCompaniesAsync(DiscProfilesContext sql, IMongoDatabase mongo)
{
    Console.WriteLine("Migrating companies...");

    var rows = await sql.companies.ToListAsync();
    var docs = rows.ConvertAll(c => new CompanyDocument
    {
        Id = c.id,
        Name = c.name,
        Location = c.location,
        BusinessField = c.business_field
    });

    var col = mongo.GetCollection<CompanyDocument>("companies");
    await col.DeleteManyAsync(_ => true);
    if (docs.Count > 0)
        await col.InsertManyAsync(docs);

    Console.WriteLine($"  → {docs.Count} companies migrated.");
}
```

Figure 34: Migration

Then we set up the ef db context for sql database and the mongo db client.

and call all the functions where we migrate all entities from sql to mongo.

this is how the function looks like, we convert from sql entities to mongo documents,
we get all the rows from companies in a list and convert them to a mongo document
Then it gets all collections inside the company document and empties it before inserting the data. That makes sure you can keep running the migrator without getting duplicate data.
This structure for the function is the same for all the other entities.

This is a mongodb modelclass.

```
public class CompanyDocument
{
    [BsonId]
    1 reference | Harun Hursid, 2 days ago | 1 author, 1 change
    public int Id { get; set; }

    [BsonElement("name")]
    1 reference | Harun Hursid, 2 days ago | 1 author, 1 change
    public string Name { get; set; } = string.Empty;

    [BsonElement("location")]
    1 reference | Harun Hursid, 2 days ago | 1 author, 1 change
    public string? Location { get; set; }

    [BsonElement("business_field")]
    1 reference | Harun Hursid, 2 days ago | 1 author, 1 change
    public string? BusinessField { get; set; }
}
```

Figure 35: Mongodb modelclass

It defines how a company document should look in mongo db and how C# should serialize and deserialize the fields.
The “BsonId” is telling mongo db that this is the primary key, similar to pk in sql.
The “BsonElement” is how it maps the fields.

When running the migrator it does the job pretty fast, it takes like a second to complete.

3. Document database

3.1. Intro to document databases

MongoDB is a NoSQL database that stores data in flexible, JSON-like documents instead of fixed tables like in a traditional SQL database. This means you do not need a strict schema, and the data structure can easily change as your application grows.
In SQL databases, data is often split into many tables and connected with JOINs. MongoDB keeps related data together in one document, which can make the design simpler and easier to work with.
Even though MongoDB works differently from SQL, it still supports important features such as indexing, transactions, and advanced queries. Overall, it offers a more flexible way to store data, especially for modern applications.[4]

3.2. Database design + features like indexes, PKs, constraints, etc.

In our project, we chose to design MongoDB so that it mirrors the structure of our SQL database. SQL Server is our primary database, while MongoDB works as an alternative storage layer where we can access the same data in a more flexible, document-based way. Because of this, each SQL table has a matching MongoDB collection with almost the same fields. This keeps things simple, makes migration easy, and lets us reuse the same DTOs for both SQL and Mongo CRUD operations.

We also kept the same primary key values from SQL and used them as the '_id' field in MongoDB. This makes it very easy to match records across SQL and MongoDB. MongoDB automatically creates an index on '_id', so we get fast lookups without having to configure anything ourselves.

For relationships, instead of embedding documents inside each other which is common in MongoDB, we chose to keep the same structure as SQL by storing foreign key fields such as 'employee_id' or 'project_id' directly in the documents. This makes it straightforward to rebuild relationships when querying MongoDB. Even though MongoDB does not enforce

[4] https://www.mongodb.com/resources/basics/databases/nosql-explained/nosql-vs-sql

foreign key constraints, this works well because the data comes from SQL, which already guarantees that everything is consistent.

We did not add extra indexes or constraints beyond the automatic '_id' index, since our data set is small and MongoDB is not the primary system. If this were a large production system, we could add more indexes later, but for our use case the simple setup is enough.
Overall, the MongoDB design is intentionally simple and closely aligned with SQL, which makes the migration reliable, keeps the data consistent, and reduces the amount of special logic we need to maintain.

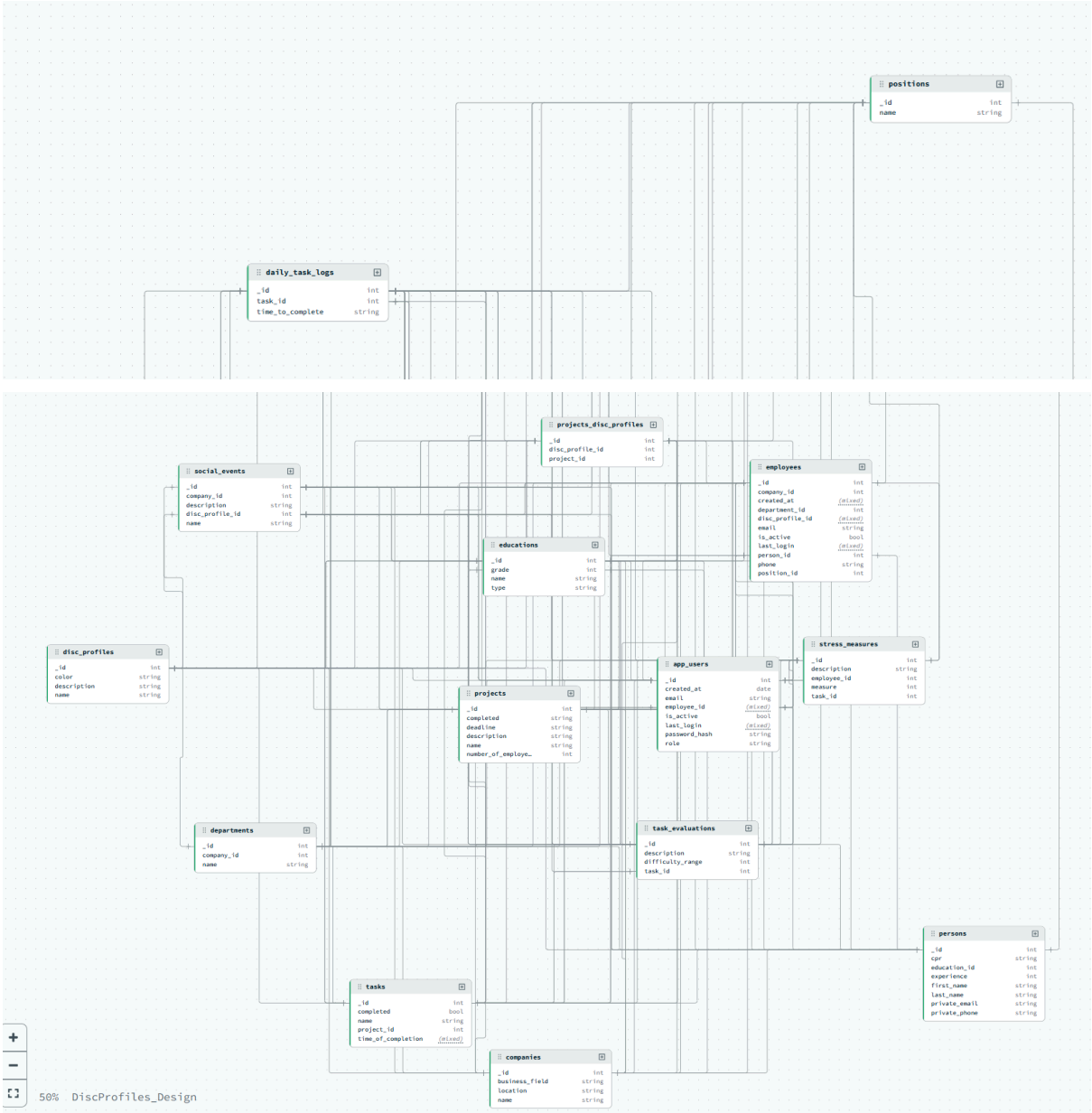

Figure 36: MongoDB database design

This is how our database design looks like. As you can see on the pictures above we have a lot of connections.

```
_id: 11
email : "user1@test.com"
phone : "123456"
company_id : 1
person_id : 17
department_id : 1
position_id : 1
disc_profile_id : null
created_at : 2025-11-23T19:56:33.210+00:00
is_active : true
last_login : 2025-11-23T19:56:33.210+00:00
```

Figure 37: Mongodb document

This is how it looks in our compass when we look inside the 'employee collection' and take a look at a 'mongo document' it looks just like our sql table with the same fields and the fk keys from sql is just a regular int field as described earlier.

3.3. Description of the CRUD application for the document database

For our CRUD application, we decided to keep everything inside the same api. This means the api can handle CRUD operations for both SQL Server and MongoDB, simply by using different controllers for the different database endpoints. This keeps the system easy to work with and avoids splitting the project into multiple services.

The design for MongoDB follows the same generic architecture we use for the relational database. The main difference is that instead of using SQL entities, we created separate MongoDocument classes that represent the collections in MongoDB. These documents can be mapped to the exact same DTOs that we already use for SQL, which keeps our API consistent across both databases.

We also created a Generic Repository and a Generic Service specifically for MongoDB. They work almost the same way as our generic repository and service for SQL, but internally they use the MongoDB driver and IMongoCollection<TDocument> instead of Entity Framework.

Each Mongo controller simply depends on this generic service and makes the CRUD operations available through the api.

This setup allows us to reuse a lot of code, maintain a consistent structure across both database systems, and keep everything inside a single API without adding unnecessary complexity.

```
5 references | Harun Hursid, 4 days ago | 1 author, 1 change
public class CompanyDocument
{
    [BsonId]
    2 references | Harun Hursid, 4 days ago | 1 author, 1 change
    public int Id { get; set; }

    [BsonElement("name")]
    2 references | Harun Hursid, 4 days ago | 1 author, 1 change
    public string Name { get; set; } = string.Empty;

    [BsonElement("location")]
    2 references | Harun Hursid, 4 days ago | 1 author, 1 change
    public string? Location { get; set; }

    [BsonElement("business_field")]
    2 references | Harun Hursid, 4 days ago | 1 author, 1 change
    public string? BusinessField { get; set; }
}
```

Figure 38: Mongodb dokument in C#

This is how a document class looks like. It's similar to the ones used in our migrator.

```
2 references | Harun Hursid, 4 days ago | 1 author, 1 change
public async Task<List<TDocument>> GetAllAsync() =>
    await _collection.Find(Builders<TDocument>.Filter.Empty).ToListAsync();
```

Figure 39: Mongodb TDokument example

This is an example of where we use the <TDocument> as described earlier.

4. Graph database

4.1. Intro to Graph databases

In a graph database we don't work with tables and rows like in a normal relational database. Instead we have nodes and relationships.
Nodes are our objects, for example an Employee or a person. Both nodes and relationships can have small pieces of data attached to them (key–value pairs like id, name, email). This way of modelling is called the property graph model.

The important thing in a graph database is the relationships. Connections like EMPLOYEE -[HAS _ PERSON]-> PERSON directly in the database and are very fast to follow. We don't need to do heavy JOINs like in SQL. Because of that, graph databases work really well when the data has lots of connections. It works faster and more economically, and relationships are already predetermined based on connected nodes.

In our project we use Neo4j. It is a native graph database with ACID transactions and its own query language called Cypher. Cypher is a bit like SQL, but for graphs. With Cypher we can write queries that describe patterns in the graph, for example: "find all employees that has person" as seen in the figure.

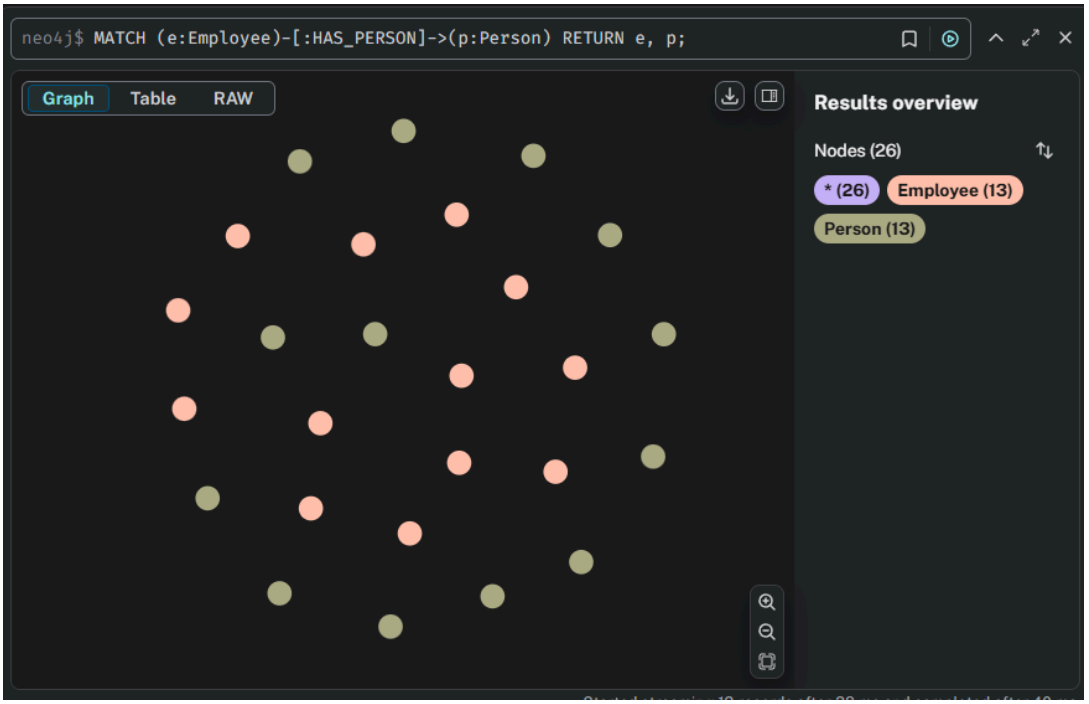

Figure 40: Graph desktop example

4.2. Database design + features like indexes, PKs, constraints, etc.

Data is stored in the nodes and edges, where nodes contain the information and the edges define the relation between 2 nodes. The node can have a label like in our project employee. The connected data is stored as connected data.
Nodes are the entities in the graph.They hold any number of attributes called properties, and the nodes can be tagged with labels[5].

Our graph model is basically our relational model, shown as a network of circles and arrows instead of tables. In the figure below, the company sits in the middle. Around it you can see the employees with the work mail, the departments like HR and the person nodes with the employees private info such as private phone and mail. Each employee has an arrow to the company with the "WORKS_FOR" relationship, and an arrow to the department with "IN_DEPARTMENT" and an arrow to a person node with HAS_PERSON. By just following the arrows, you can read the structure for the whole database.

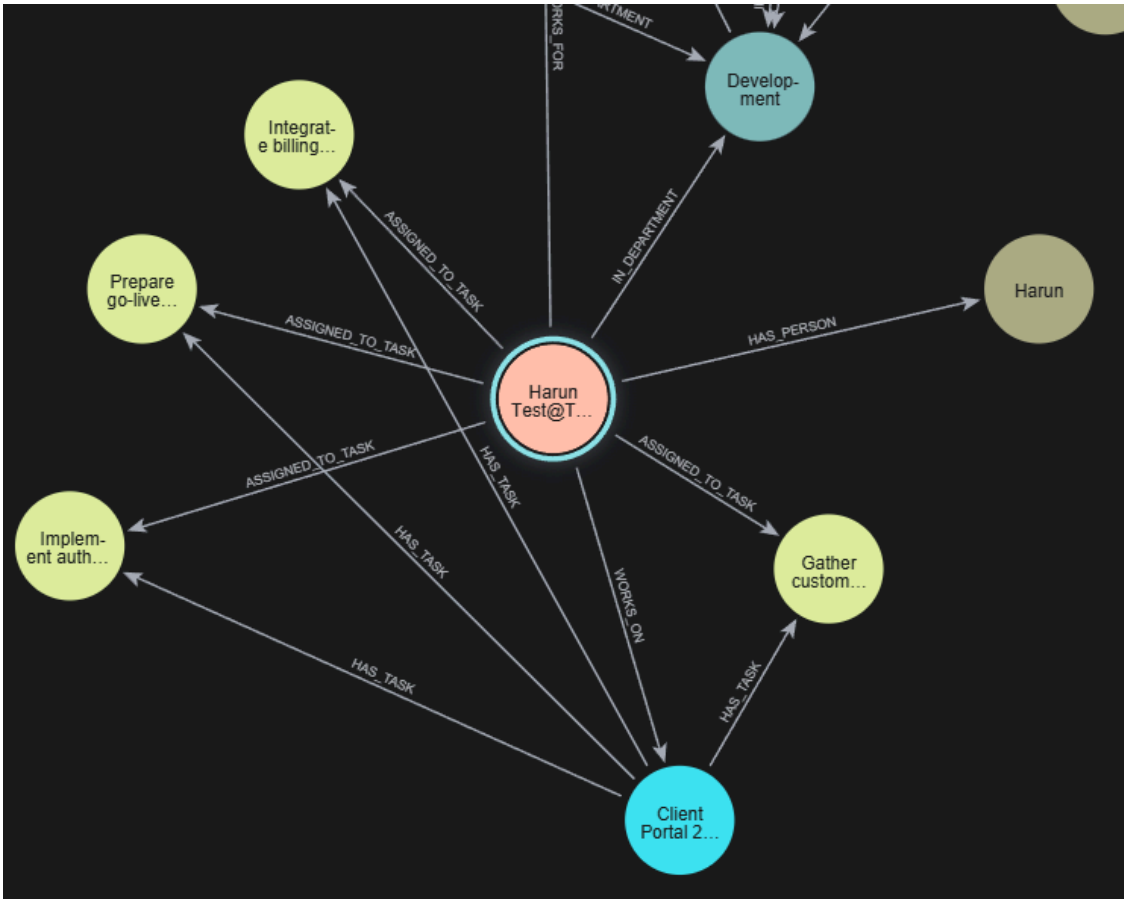

Figure 41: Graph relationships

[5] Graph database powerpoint from lecture - Graph_DB lecture

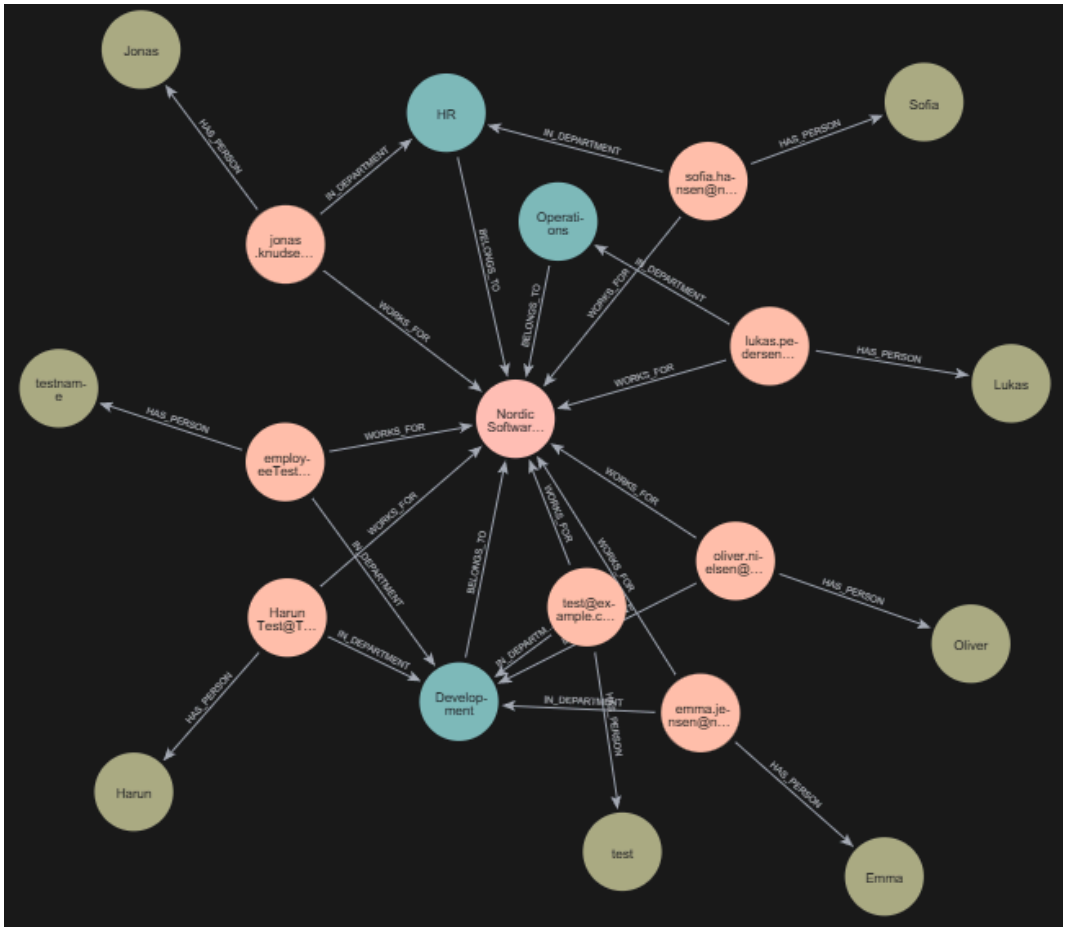

Figure 42: Graph relationships

This is pretty close to how we would explain it in normal language. The nice thing is that we don't need join tables like in SQL, because links between things are stored directly as relationships in the graph.

4.2.1 Primary keys

Even though we don’t have tables, we still need a way to uniquely identify each node, just like a primary key in SQL.

For that we use an id field on each main node type, employee, person, company etc. The idea is that there should only be one node of each label with a given id. This is what we treat as our “primary key” in the graph.

4.2.2 Indexes

To make lookups fast, Neo4j builds indexes on these id fields. That means when we run a query Neo4j uses the index to find the right node quickly instead of scanning all employees. After that it can follow the relationship out from that node to get the next node. In our project we can use an employee with the query "MATCH (e:Employee {id: X}), where it finds the right node and then extends the node to get to the company, departments, projects, tasks and so on[6].

4.2.3 Constraints

Since we only have access to the community version, our uniqueness to our project only sticks to the id's. Neo4j supports more advanced constraint types, but those require the enterprise edition[7].

```
neo4j$ CREATE CONSTRAINT unique_person IF NOT EXISTS FOR (p:Person) REQUIRE p.id IS UNIQUE;
Added 1 constraint

CREATE CONSTRAINT unique_employee IF NOT EXISTS
FOR (e:Employee) REQUIRE e.id IS UNIQUE;

No changes, no records
00NA0: Note: successful completion - index or constraint already exists
```

Figure 43: Create constraint graph desktop

```
neo4j$ CREATE CONSTRAINT unique_employee IF NOT EXISTS FOR (e:Employee) REQUIRE e.id IS UNIQUE; CREATE CONSTRAINT unique_person IF NOT
CREATE CONSTRAINT unique_employee IF NOT EXISTS
CREATE CONSTRAINT unique_person IF NOT EXISTS
CREATE CONSTRAINT unique_company IF NOT EXISTS
CREATE CONSTRAINT unique_project IF NOT EXISTS
CREATE CONSTRAINT unique_task IF NOT EXISTS
```

Figure 44: Create constraint graph desktop

[6] https://neo4j.com/docs/cypher-manual/current/indexes/
[7] https://neo4j.com/docs/cypher-manual/current/constraints/

4.3. Description of the CRUD application for the graph database

The web API for the graph database is basically the same as for our relational database. We still use ASP.NET controllers, DTO's and services. The big difference is only the data layer, instead of Entity Framework and SQL the graph part uses the Neo4j .NET driver and Cypher queries.

For the CRUD on employees, projects etc. we keep using SQL controllers. They talk to the relational database through EF Core. When we want data to appear in the graph as well, we call into the service, which reads the same data from SQL and mirrors it into Neo4j.
Here we see cypher-query where we read employees and the company from the graph and mapping it to EmployeeCompanyDTO.

```
var cursor = await session.RunAsync(@"
    MATCH (e:Employee)-[:WORKS_FOR]->(c:Company)
    RETURN e.id AS employeeId, c.name AS companyName
");
```

Figure 45: Cypher in project

```
var cursor = await session.RunAsync(@"
    MATCH (e:Employee {id: $employeeId})-[:WORKS_ON]->(p:Project)-[:HAS_TASK]->(t:Task)
    RETURN e.id AS employeeId,
           p.name AS projectName,
           t.id AS taskId,
           t.name AS taskName
", new { employeeId });
```

Figure 46: Cypher in project

In the figure above, the query first finds the employee node with the given id, then follows the WORKS_ON relationship to the employees' projects, and from each project follows HAS_TASK to its task. The method then maps each record from NEo4j into an EmployeeDto with employee id, project name and task info and returns the list.

We also use MirrorEmployeeFromSqlAsync. When a new user is created is AppUserController.Create, we first store everything in SQL. At the end we call MirrorEmployeeFromSqlAsync(employee.id). This method reads the employee and its related data from SQL and mirrors it into Neo4j as nodes and relationships. In that way, the graph database is updated automatically every time we create a new user, or update the user.

```
try
{
    await _graphEmployeeService.MirrorEmployeeFromSqlAsync(employee.id);
}
```

Figure 47: Mirror method in code

The same idea is used in AdminAssignmentsController, when an admin assigns an employee to a project or a task. After the change is saved in SQL, we call the same mirror method again, so Neo4j is updated with the new relationships. Thus making SQL the "source of truth or Ground of truth" and the graph is continuously kept in sync.

Updates and deletes still go through the SQL API first. After changing data in the relational database we can call the sync method in GraphEmployeeService again, which will update the corresponding nodes and relationships in Neo4j. So overall the HTTP API and the basic CRUD operations look almost identical to the RDBMS solution; the difference is that some endpoints now use Cypher and the graph database to read connected data in a way that fits the graph model better.

5. Cloud

As mentioned earlier we have to host our databases in the cloud. For MongoDB and Neo4j it was straightforward to do because they have their own native sites to do the hosting on a free plan. For MongoDb we use MongoDB Atlas to host our Database in a Cluster that we can access in the desktop app called compass. for Neo4j we used Neo4j aura where we host the database.

All these are then used to connect to our projects via connection string, which since we had different logins and passwords made use of .env files, which we explained further up in the report.

6. Conclusion

The goal of this project was to build a system that can help organisations understand and improve collaboration using DISC profiles. To support this, we designed and implemented a relational database in SQL Server as our main data source, and then added a document database (MongoDB) and a graph database (Neo4j) as alternative ways to work with the same data. This gave us a running solution, but also a chance to compare three very different database models on the same domain.

SQL Server is the core of the system. Here we worked with ER modelling, normal forms, primary and foreign keys, indexes, constraints, triggers, stored procedures, functions and views. We made some early mistakes, for example starting too quickly on the physical model and storing derived values, but this helped us understand why normalisation and a clear conceptual model are important. The final schema is more focused, easier to read and better aligned with the business logic around employees, projects and tasks. In practice, the relational database is the best choice for our serious business data, it gives strong consistency, clear constraints and good support for complex reporting through SQL and stored procedures.

MongoDB and Neo4j were added mainly to explore other ways of representing the same information. In MongoDB we mirrored the SQL structure closely, reusing ids and DTOs, and used a small migrator app to keep the data in sync. MongoDB's document model is more flexible and would make it easy to change the structure of entities over time without heavy migrations. In our project, MongoDB does not replace SQL Server, but shows how a document store can act as a secondary store for the same data, for example for lighter read scenarios, experiments or future reporting, without enforcing foreign key constraints.

Neo4j takes a different angle by focusing on relationships instead of tables. In the graph model we described how employees work for a company, belong to a department and are connected to projects and tasks. Here Cypher makes it easier to express queries that follow several hops in the graph, for example "which tasks belong to projects that a specific employee works on?". The GraphEmployeeService and the mirror method keep the graph updated based on changes in SQL. Compared to SQL, Neo4j is more natural when the question is mostly about "who is connected to what and how?", but it requires extra work to keep it in sync and does not enforce as many constraints as the relational database.

Seen together, the three databases show clear strengths and trade-offs. The relational database is best for strict, central business data. The document database is best when we need flexible structures and fast, simple reads without a heavy schema. The graph database is best when relationships are the main thing we care about, and when queries become hard to express with joins. In our solution we deliberately kept SQL Server as the single source of truth and used MongoDB and Neo4j as additional views on the same data. Overall, the project has given us practical experience with how different database technologies can be combined in one system, and a better understanding of when a relational, document or graph database is the best fit.

7. References

1. Microsoft dokumentation about Sql injection
   Website link:
   https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/ef/security-considerations
2. MongoDb dokumentation
   Website link:
   https://www.mongodb.com/resources/basics/databases/nosql-explained/nosql-vs-sql
3. Neo4j dokumentation
   Website link:
   https://neo4j.com/docs/cypher-manual/current/indexes/
   https://neo4j.com/docs/cypher-manual/current/constraints/
4. Disc profiles
   Website link:
   https://www.discprofile.com/disc-styles
5. Relational database
   Website link:
   https://www.ibm.com/think/topics/relational-databases
   https://www.red-gate.com/blog/chen-erd-notation
   https://www.red-gate.com/blog/crow-s-foot-notation